

Adaptive Multimodal Fusion for Social Media Content Classification using PySpark

S. M. SAIDUR RAHMAN (22101451) , SHAYONTO RAYHAN (22101651)
CSE761 Project Report

Abstract—Fake news on social media is often multimodal, and real posts do not always have both text and image available. We build a distributed PySpark pipeline for the Fakeddit dataset (fine-grained, 6-way classification) that performs ETL, GPU-batched feature extraction, and MLlib/PyTorch classification, and use it to compare a concatenation baseline against several adaptive, gated fusion designs. A confidence-weighted gate trained separately from the classifier does not beat simple concatenation – we trace this to the gate never being optimized against the downstream loss. Training the gate jointly with the classifier, end-to-end, fixes this: our final model, adaptive-joint, beats the concatenation baseline in all three tested conditions (both modalities present, image missing, text missing), with the largest gain on clean data (65.1% vs. 56.2% accuracy).

I. INTRODUCTION

Misinformation on social platforms is often multimodal: a real image paired with a misleading caption, or an old photo reused alongside a false claim about a current event. In both cases, neither the text nor the image is false on its own – only the pairing is. A text-only or image-only detector cannot catch this, since the misleading signal exists only in how the two relate. Making this harder, real posts do not always arrive with both modalities intact – in our own data collection, nearly 40% of sampled posts had a broken or unreachable image link, so any practical system must handle a missing modality, not just fuse two always-present inputs.

This is also a big-data problem, not just a modeling one. A typical deep learning pipeline runs on one machine in one process, with no built-in way to scale across a cluster. At the scale of millions of posts – each needing an image download, an image embedding, and a text embedding – this becomes a bottleneck. We build our pipeline on Apache Spark instead: Spark splits the data across workers automatically, so cleaning and feature extraction run in parallel rather than one row at a time; MLlib’s feature tools operate directly on this distributed data; and a GPU-batched Spark UDF computes image embeddings across partitions rather than in a single sequential loop.

Since modalities are not always available, always trusting both equally is risky. This motivates our *adaptive* fusion approach: the model learns, per sample, how much to rely on each modality, rather than weighting both the same way every time. We evaluate fusion both on clean data and when a modality is missing entirely.

Contributions: (1) a distributed Spark pipeline for multimodal fake-news classification; (2) a systematic comparison of a concatenation baseline against multiple adaptive fusion

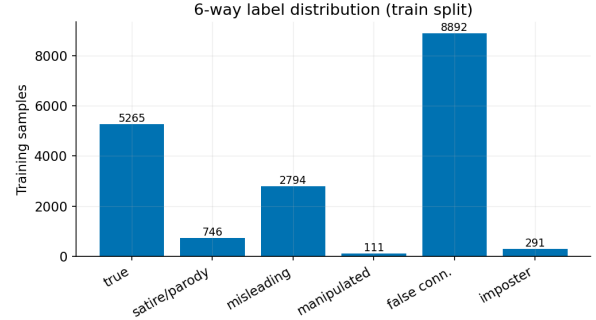


Fig. 1: 6-way label distribution, training split.

designs; (3) a diagnosed failure mode in confidence-gated fusion and a demonstrated fix via joint end-to-end training.

II. RELATED WORK

Fakeddit [1] introduced a $\sim 1\text{M}$ -sample, multimodal, multi-granularity (2-/3-/6-way) benchmark for fake news detection sourced from Reddit. Gated multimodal fusion – a learned, per-sample weighting between modalities rather than fixed concatenation – traces to the Gated Multimodal Unit [2], which our adaptive gate follows in spirit.

III. DATASET

We use the Fakeddit `multimodal_only_samples` release restricted to the 6-way label (*true*, *satire/parody*, *misleading content*, *manipulated content*, *false connection*, *imposter content*). We draw a stratified sample (target 30,000/5,000/5,000 train/val/test) and retain only posts whose image URL still resolves; 2019-era Reddit image links suffer substantial rot, so the final usable sample is 18,099/1,401/1,443 (60.3% image-download success).

The label distribution is heavily imbalanced (Fig. 1): *false connection* alone accounts for 49.1% of training data, while *manipulated content* (0.6%) and *imposter content* (1.6%) are extremely rare. Because of this, simply guessing the majority class every time would already score about 49% accuracy. So the accuracy numbers reported later should be judged against that $\sim 49\%$ floor, not against 100%.

IV. METHODOLOGY

Pipeline Raw data is cleaned and stratified-sampled, then loaded into Spark; from there, parallel text/image feature extraction (Word2Vec; frozen ResNet-18 via a GPU-batched

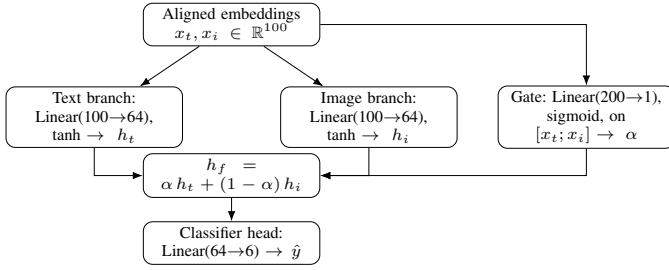


Fig. 2: Adaptive-joint fusion architecture: text and image branches combined via a gate α trained jointly with the rest of the network, so α receives direct gradient feedback from the classification loss.

mapInPandas UDF) and PCA alignment produce a common 100-dim embedding per modality before reaching the fusion step. Fig. 2 details our main contribution: the **adaptive-joint** architecture, the only fusion design in which the gate α is trained end-to-end with the rest of the network rather than being computed by separately pre-trained, frozen models.

Fusion architectures compared Each entry below states its input, its classifier, and which training set it uses – clean (complete rows only) or augmented (includes rows with a modality zeroed out; see below).

- **static-concat** (baseline): the two 100-dim embeddings concatenated (200-dim input) $\rightarrow$ a single multinomial LogisticRegression (linear, no hidden layer). Trained on the *clean* set.
- **static-concat-aug**: identical input and classifier to static-concat, but trained on the *augmented* set instead – isolates whether any gap against the adaptive models comes from the architecture or just from what data it saw during training.
- **adaptive-fused**: two separately pre-trained, frozen unimodal MultilayerPerceptronClassifiers (one per modality) each output a confidence score; these combine into a per-sample gate weight α . Classifier input is the α -weighted concatenation plus α itself (201-dim) $\rightarrow$ LogisticRegression. Trained on the *augmented* set.
- **adaptive-routed**: not a separately trained model – a rule applied at inference on top of the models above: use adaptive-fused’s classifier when both modalities are present, otherwise defer directly to the matching frozen unimodal MLP.
- **rf-concat**: the same 200-dim concatenated input as static-concat, but with a nonlinear RandomForestClassifier in place of the linear model. (Spark MLlib’s GBTClassifier does not support multiclass, so RandomForestClassifier is used instead.) Trained on the *clean* set.
- **adaptive-joint** (final model): the only non-Spark, non-linear model – one PyTorch network with a text branch, an image branch, and a gate that computes α directly from the raw 200-dim concatenation, all trained together

TABLE I: Experimental configuration

Text encoder	Word2Vec, 100-dim (frozen)
Image encoder	ResNet-18, ImageNet-pretrained, frozen 512 $\rightarrow$ 100-dim via PCA
Modality dropout	15% image-dropped + 15% text-dropped
Non-joint classifiers	Spark MLlib LogisticRegression
Joint network	100 $\rightarrow$ 64 (text) + 100 $\rightarrow$ 64 (image) + gate (200 $\rightarrow$ 1, sigmoid) + head (64 $\rightarrow$ 6)
Hardware	Adam, lr = 10^{-3} , 30 epochs, batch 256
Metrics	Kaggle, 2 $\times$ NVIDIA T4, Spark local[*]
Test conditions	Accuracy, weighted F1
	Full, image-missing, text-missing

end-to-end with a single loss, so α gets direct gradient feedback on whether its weighting choices helped the actual prediction. Trained on the *augmented* set.

The two adaptive designs differ in how α is used to fuse $x_t, x_i \in \mathbb{R}^{100}$ (the text and image embeddings). adaptive-fused concatenates the α -weighted *raw* embeddings directly as the classifier input:

$$f_{\text{fused}} = [\alpha x_t ; (1 - \alpha) x_i ; \alpha], \quad (1)$$

where α comes from two independently pre-trained, frozen confidence models. adaptive-joint instead first projects each embedding into a 64-dim branch representation (h_t, h_i), computes α from a gate trained jointly with the rest of the network, and combines the branches directly:

$$h_f = \alpha h_t + (1 - \alpha) h_i. \quad (2)$$

Because α and the branches in adaptive-joint are updated by the same backpropagated loss, α receives direct feedback on whether it helped the final prediction – unlike adaptive-fused, where α comes from models trained on a separate, disconnected objective.

The *augmented* training set adds modality dropout: for 15% of rows the image embedding is zeroed out, for another 15% the text embedding is zeroed out (labels are untouched), so these classifiers see text-heavy and image-heavy examples during training, not only complete ones. All test-set evaluation, in every condition, always uses the clean test set (or a copy of it with one modality zeroed, for the missing-modality conditions) – the augmented data is never used for evaluation, only for training.

V. EXPERIMENTAL SETUP

Table I summarizes the encoders, classifiers, and training configuration used throughout.

VI. RESULTS

Table II summarizes all runs. **adaptive-joint outperforms the baseline in every condition**: full (+8.9 points accuracy), image-missing (+24.3 points), and text-missing (+4.6 points) – the only model to do so consistently across all three.

Gate behavior The learned α has standard deviation 0.088 on the test set (Fig. 4) – the joint gate does vary its trust between modalities per sample, though modestly rather than dramatically.

TABLE II: Accuracy / weighted F1 across all evaluated conditions (6-way)

Model	Full		Image missing		Text missing	
	Acc.	F1	Acc.	F1	Acc.	F1
text-only	0.428	0.448	—	—	—	—
image-only	0.514	0.530	—	—	—	—
static-concat (baseline)	0.562	0.569	0.130	0.169	0.529	0.537
static-concat-aug	0.565	0.571	0.148	0.191	0.526	0.534
rf-concat	0.547	0.549	0.558	0.497	0.474	0.362
adaptive-fused	0.518	0.514	0.223	0.275	0.548	0.554
adaptive-routed	0.518	0.514	0.428	0.448	0.514	0.530
adaptive-joint (final)	0.651	0.657	0.373	0.424	0.575	0.563

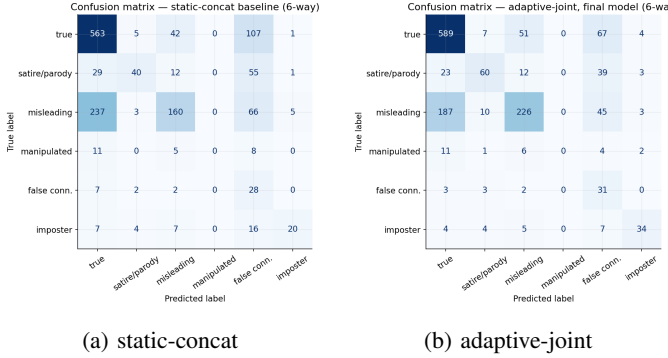
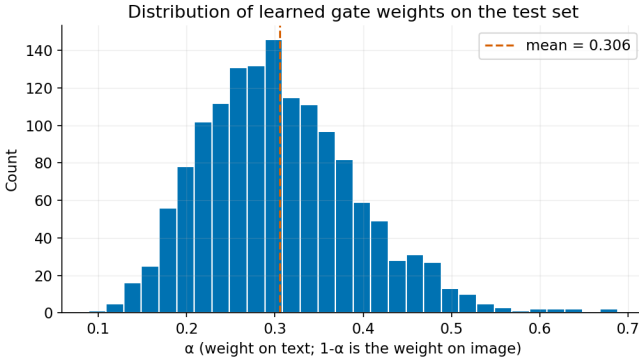


Fig. 3: Confusion matrices, baseline vs. final model (clean data).

Fig. 4: Distribution of the learned gate weight α on the test set.

VII. DISCUSSION

Why the non-joint gate loses on clean data adaptive-fused’s feature vector scales each embedding by a per-sample α derived from two *independently* trained, frozen confidence models – never optimized against the downstream classification loss. A linear classifier cannot fully correct for this: undoing a per-dimension multiplicative distortion requires interaction terms a linear model cannot represent. static-concat instead always presents the full, unscaled signal and lets one globally-optimized set of coefficients decide feature trust – a more statistically efficient strategy at this data scale, which is why it beats every non-joint adaptive variant on clean data.

Why joint training fixes it Training the gate and classifier together gives α direct gradient feedback tied to whether it actually helped the final prediction, rather than to its own, disconnected confidence objective. This is the only architectural change that lets an adaptive design beat the baseline, and it does so in all three conditions, not only on clean data.

Why static-concat is naturally robust With a linear classifier, concatenation lets each modality’s contribution be computed independently ($w_{\text{text}} \cdot x_{\text{text}} + w_{\text{image}} \cdot x_{\text{image}}$); zeroing one modality’s features zeroes its term without retraining. This explains why static-concat-aug (trained on the same modality-dropout data as adaptive-fused) performs almost identically to plain static-concat – the robustness comes from the architecture, not from training exposure.

VIII. LIMITATIONS AND FUTURE WORK

- **Scale** Experiments run on a stratified sample, not the full $\sim 1\text{M}$ -row Fakeddit release; the pipeline is architected to scale unchanged.
- **Class imbalance** *manipulated content* (111 train examples) and *imposter content* (291) are too rare for reliable per-class conclusions; overall accuracy is dominated by the majority class.
- **Calibration under partial (not missing) information** is not separately measured – only the fully-missing-modality case is tested.
- **Distributed joint training** The joint network trains as a single-process PyTorch job; Spark’s `TorchDistributor` would extend this to a multi-node cluster launch, unnecessary at the current data scale.
- **Routing + joint hybrid** adaptive-routed and adaptive-joint win in complementary regimes (image-missing floor vs. overall ceiling); combining them – joint model when both modalities are present, routed fallback otherwise – is a natural next step.

IX. CONCLUSION

We present a distributed PySpark pipeline for fine-grained multimodal fake news classification and a systematic evaluation of fusion strategies under clean and missing-modality conditions. Naive confidence-gated fusion does not outperform simple concatenation; we trace this to a lack of end-to-end optimization of the fusion gate, and show that joint training

resolves it, producing the only fusion variant in this study to beat the concatenation baseline across all evaluated conditions.

REFERENCES

- [1] K. Nakamura, S. Levy, and W. Y. Wang, “r/Fakeddit: A New Multimodal Benchmark Dataset for Fine-grained Fake News Detection,” in *Proc. LREC*, 2020.
- [2] J. Arevalo, T. Solorio, M. Montes-y-Gómez, and F. A. González, “Gated Multimodal Units for Information Fusion,” in *ICLR Workshop*, 2017.
- [3] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. CVPR*, 2016.
- [4] Apache Spark MLlib documentation. [Online]. Available: <https://spark.apache.org/docs/latest/ml-guide.html>